

Slide 1: Usage Service — Collection: usagereadings (Kỹ thuật tối ưu hóa lưu trữ)

Lời dẫn: "Tiếp theo, em xin trình bày về linh hồn của hệ thống — nơi lưu trữ hàng tỷ chỉ số điện — đó là Collection *usagereadings* trong MongoDB."

- **Ý 1 (Bucket Pattern):** "Thưa thầy cô, thay vì lưu mỗi lần đo là một bản ghi đơn lẻ gây bùng nổ dữ liệu, em đã áp dụng kỹ thuật **Bucket Pattern**. Mỗi ngày, mỗi đồng hồ chỉ tương ứng với **duy nhất 1 Document**. Toàn bộ 96 lần đo trong ngày được đẩy vào mảng **Embedded Array** tên là *readings*."
 - **Ý 2 (Tại sao chọn cách này?):** "Việc này giúp hệ thống đạt tốc độ đọc cực nhanh vì chỉ cần đúng **1 Query** là có thể vẽ ngay được biểu đồ phụ tải cả ngày của khách hàng mà không cần phép JOIN phức tạp. Đồng thời, việc lưu **GeoJSON Location** ngay tại đây giúp hệ thống thực hiện các truy vấn không gian (Geospatial) để phát hiện sự cố theo vùng địa lý ngay tức thì."
 - **Ý 3 (LSM-Tree):** "Với khối lượng dữ liệu lên đến hơn 35.000 bản ghi mỗi hộ/năm, em chọn MongoDB với Engine **LSM-Tree** để tối ưu hóa tốc độ ghi tuần tự, đảm bảo hệ thống không bị nghẽn khi hàng triệu đồng hồ cùng gửi dữ liệu về."
-

Slide 2: Usage Service — Monthly Summary & Meter Baselines (Phân tích dữ liệu nâng cao)

Lời dẫn: "Dữ liệu lớn chỉ có giá trị khi chúng ta phân tích được nó. Slide này mô tả cách hệ thống 'học' thói quen người dùng để phát hiện sự cố."

- **Ý 1 (Meter Baselines):** "Bảng *meterbaselines* lưu trữ 'dấu vân tay năng lượng' của từng hộ dân, bao gồm mức tiêu thụ trung bình (μ) và độ lệch chuẩn (σ). Những con số này không phải là hằng số mà được cập nhật liên tục."
 - **Ý 2 (Aggregation Pipeline):** "Để phát hiện bất thường, em sử dụng kỹ thuật **Aggregation Pipeline** — bộ não tính toán của MongoDB. Thay vì lấy dữ liệu về Backend xử lý, em đẩy trực tiếp công thức **Z-Score** xuống Database. Sử dụng các toán tử như $\$avg$ và $\$stdDevPop$, hệ thống sẽ tính toán độ lệch ngay trên luồng dữ liệu đang chảy."
 - **Ý 3 (Công thức Z-Score):** "Nếu chỉ số Z-Score vượt ngưỡng cho phép, hệ thống sẽ tự động coi đó là một sự cố và đẩy kết quả vào hàng đợi cảnh báo. Cách làm này giúp giảm tải cho Server và tận dụng tối đa sức mạnh tính toán song song của Database."
-

Slide 3: Notification Service — Collection: notifications (Luồng hoạt động End-to-End)

Lời dẫn: "Cuối cùng là kết quả đầu ra của quy trình phân tích — Collection notifications, nơi quản lý các cảnh báo thông minh."

- **Ý 1 (Phân loại cảnh báo):** *"Hệ thống của em không chỉ báo lỗi chung chung mà phân loại rõ ràng 3 kịch bản thực tế:
 1. **Blackout:** Mất điện hoàn toàn (Usage = 0).
 2. **Theft:** Nghi ngờ gian lận (Z-score âm cực lớn).
 3. **Anomaly:** Quá tải hoặc rò rỉ điện (Z-score dương cao)."
- **Ý 2 (Luồng End-to-End):** "Hãy nhìn vào luồng hoạt động tổng thể ở phía dưới: Từ thiết bị Smart Meter gửi dữ liệu -> Usage Service ghi vào MongoDB -> Aggregation Pipeline phân tích Z-Score -> Phát hiện bất thường -> Cuối cùng Notification Service sẽ tạo một thông báo và gửi ngay email cho Grid Operator (Người vận hành lưới điện)."
- **Ý 3 (Giá trị thực tiễn):** "Việc lưu trữ các thông báo này trong NoSQL giúp hệ thống có cấu trúc linh hoạt, dễ dàng mở rộng thêm các loại cảnh báo mới trong tương lai mà không cần thay đổi cấu trúc bảng như SQL truyền thống."

\Các trường hợp

- **(Chế độ 2)** Một hộ dân đột ngột sử dụng thiết bị công suất cực lớn không khai báo (ví dụ: máy hàn công nghiệp, dàn đào tiền ảo, hoặc bị chạm chập thiết bị trong nhà).
- **Ý nghĩa:** Test khả năng phát hiện lỗi đơn lẻ của hệ thống. Dashboard sẽ chỉ hiện 1 chấm cam cảnh báo tại đúng nhà đó.
- **(Chế độ 3)** là tình huống rất quan trọng trong vận hành lưới điện. Dưới đây là các ví dụ thực tế nhất:

1. Nắng nóng đỉnh điểm (Heatwave)

Đây là trường hợp phổ biến nhất tại Việt Nam. Khi nhiệt độ ngoài trời vọt lên trên 40 độ C vào tầm 12h trưa hoặc 20h tối, **tất cả các hộ dân trong khu vực đồng loạt bật điều hòa ở mức thấp nhất.**

- **Hậu quả:** Phụ tải toàn trạm biến áp vọt lên mức đỏ, dễ gây cháy nổ trạm hoặc nhảy Aptomat tổng. Hệ thống của bạn sẽ thấy hàng trăm hộ cùng lúc có Z-Score cao.

2. Giờ cao điểm sinh hoạt (Evening Peak)

Tầm 18h30 - 19h30 tối, khi mọi người đi làm về:

- Đồng loạt bật bếp từ nấu cơm, bật bình nóng lạnh tắm, bật tivi, đèn chiếu sáng và điều hòa.
- Việc cộng dồn công suất của hàng trăm thiết bị này tạo ra một "cú sốc" phụ tải cho lưới điện khu vực.

3. Sự cố Sụt áp lưới điện (Voltage Sag)

Khi lưới điện gặp sự cố nhỏ gây sụt điện áp (Voltage drop), theo nguyên lý vật lý, một số loại thiết bị (như động cơ máy giặt, tủ lạnh đời cũ) sẽ **tự động tăng cường dòng điện (Ampe)** để bù đắp cho phần áp thiếu hụt nhằm duy trì công suất.

- **Kết quả:** Các đồng hồ đo sẽ ghi nhận chỉ số tiêu thụ tăng đột biến trên diện rộng ngay trước khi trạm biến áp bị ngắt hoàn toàn.

4. Khu vực có làng nghề hoặc xưởng sản xuất xen kẽ

Ở nhiều vùng, nhà dân nằm xen kẽ các xưởng may, xưởng cơ khí nhỏ.

- Khi xưởng bước vào ca sản xuất và khởi động hàng loạt máy may, máy hàn, máy cắt, phụ tải của cả khu phố đó sẽ bị kéo lên rất cao. Hệ thống của bạn sẽ cảnh báo đây là lỗi "Vĩ mô" (Macro) vì nó ảnh hưởng đến chất lượng điện của cả vùng.

5. Lễ hội hoặc Sự kiện lớn trong khu dân cư

Ví dụ một sân vận động địa phương hoặc một công viên trong khu dân cư tổ chức sự kiện âm nhạc, hội chợ. Họ sử dụng hệ thống loa đài và chiếu sáng công suất lớn chung nguồn với lưới dân sinh.

Tại sao hệ thống của bạn cần phân biệt cái này? Nếu hệ thống thấy 1 nhà cao -> Báo thợ điện đến nhà đó. Nhưng nếu thấy **100 nhà cùng cao** -> Hệ thống phải báo ngay cho trung tâm điều độ: **"Trạm biến áp này sắp cháy, dừng cho thợ đến nhà dân nữa, hãy đi kiểm tra trạm ngay!"**. Đây chính là giá trị của việc phân tích lỗi Vĩ mô._

- **Chế độ 4: Mất điện cá nhân (Individual Power Outage)**
 - **Thực tế:** Đứt dây điện vào nhà, hỏng Aptomat tổng của hộ dân, hoặc khách hàng cố tình ngắt điện để sửa chữa mà không thông báo.
 - **Ý nghĩa:** Hệ thống phát hiện tiêu thụ rớt về 0. Ngay lập tức kích hoạt Trigger SQL để chuyển trạng thái khách hàng sang **CRITICAL** và gửi lệnh công tác cho thợ điện.
-
- **Chế độ 5: Mất điện toàn khu (Area-Wide Blackout)**
 - **Thực tế:** Cháy trạm biến áp khu vực, nổ tụ điện trung thế, hoặc cắt điện luân phiên theo kế hoạch.
 - **Ý nghĩa:** Đây là tình huống khẩn cấp nhất. Hệ thống sẽ chặn hàng ngàn thông báo lẻ (để tránh làm nghẽn Server) và chỉ phát ra 1 cảnh báo đỏ rực duy nhất: **"KHU VỰC X ĐANG MẤT ĐIỆN DIỆN RỘNG"**.
 - **Chế độ 6: Bất thường nhẹ (Predictive Maintenance/Watchlist)**
 - **Thực tế:** Khách hàng quên tắt điều hòa khi đi vắng, hoặc có thiết bị điện bị rò rỉ nhẹ khiến tiêu thụ tăng cao hơn mức trung bình một chút.
 - **Ý nghĩa:** Dùng để test tính năng **Watchlist (Danh sách theo dõi)**. Các hộ này chưa đến mức báo động đỏ nhưng cần kỹ thuật viên lưu ý để kiểm tra sớm, tránh để xảy ra cháy nổ hoặc quá tải thật.

Vấn đề time seri bản ghi 15p

1. Bucket Pattern (Tối ưu hóa Time-Series)

Đây là kỹ thuật quan trọng nhất trong việc lưu trữ dữ liệu đồng hồ đo (Smart Meter).

- **Cách áp dụng:** Thay vì lưu mỗi lần đo là một Document (gây bùng nổ số lượng bản ghi), chúng ta gom tất cả các lần đo trong cùng một ngày của một đồng hồ vào **duy nhất 1 Document** với mảng readings.
- **Lợi ích:** Giảm kích thước Index, tăng tốc độ truy vấn lịch sử 24h và tiết kiệm dung lượng lưu trữ trên MongoDB Atlas.

dùng \$inc thay vì dùng hàm Sum() mỗi lần truy vấn:

1. Hiệu năng (Analogy - Ví dụ thực tế)

Hãy tưởng tượng bạn có một con lợn đất tiết kiệm:

- **Cách chúng ta đang dùng (\$inc):** Mỗi lần bạn nhét 1 đồng xu vào, bạn lấy bút ghi đè số tổng mới lên một tờ giấy dán ngoài con lợn. Khi muốn biết có bao nhiêu tiền, bạn **chỉ mất 1 giây** nhìn vào tờ giấy là xong.
- **Cách dùng hàm Sum():** Mỗi lần bạn muốn biết mình có bao nhiêu tiền, bạn phải **đập con lợn ra, ngồi đếm lại từng đồng xu một**, sau đó mới biết kết quả.

=> Với hệ thống điện, mỗi ngày có 96 bản ghi. Nếu có 1 triệu khách hàng, dùng hàm Sum() nghĩa là Database phải "đếm" lại **96 triệu** con số mỗi khi bạn load trang Dashboard. Dùng \$inc thì Database chỉ cần đọc đúng 1 con số đã ghi sẵn.

Mongo có tính nguyên tử khi update một document

3. Aggregation Pipeline (Xử lý dữ liệu phức tạp)

Thay vì xử lý logic ở code Node.js, chúng ta đẩy toàn bộ tính toán xuống tầng Database.

- **Phát hiện bất thường (Anomaly Detection):** Sử dụng `$group`, `$avg`, `$stdDevPop` để tính toán trung bình (μ) và độ lệch chuẩn (σ) của toàn hệ thống trong thời gian thực, từ đó tính toán **Z-Score** để tìm ra các hộ dùng điện bất thường.
- **Tổng hợp lịch sử hệ thống:** Sử dụng `$unwind` để "trải phẳng" mảng dữ liệu và `$group` theo mốc thời gian để vẽ biểu đồ phụ tải toàn mạng lưới.

Aggregation Pipeline trong MongoDB có thể hiểu đơn giản là một **"Dây chuyền sản xuất"**. Dữ liệu thô đi vào đầu dây chuyền, trải qua nhiều công đoạn xử lý (lọc, cắt, cộng, chia), và kết quả cuối cùng là một "sản phẩm" đã được chế biến sẵn (như biểu đồ hoặc danh sách sự cố).

Trong hệ thống của bạn, kỹ thuật này được dùng ở 3 phần quan trọng nhất:

1. Công đoạn "Tính toán Phụ tải hệ thống" (Dùng cho Biểu đồ Dashboard)

Khi bạn nhìn vào biểu đồ lên xuống của cả thành phố, Aggregation đã làm việc như sau:

- **Bước 1 (\$match):** Nhặt ra tất cả các "hộp dữ liệu" của ngày hôm nay.
- **Bước 2 (\$unwind):** Mở tung các hộp đó ra để lấy từng bản ghi 15 phút bên trong.
- **Bước 3 (\$group):** Gom tất cả các bản ghi có cùng mốc thời gian (ví dụ cùng lúc 14:00) lại một chỗ.
- **Bước 4 (\$sum):** Cộng tổng lượng điện của tất cả các hộ tại mốc 14:00 đó.
- **Kết quả:** Một danh sách các mốc thời gian và tổng điện năng để vẽ biểu đồ.

2. Công đoạn "Tầm soát sự cố" (Phát hiện Anomaly/Z-Score)

Đây là phần thông minh nhất. Nó xử lý số liệu thống kê cực nhanh:

- **Bước 1:** Tính trung bình cộng (`$avg`) và độ lệch chuẩn (`$stdDevPop`) của **toàn bộ triệu đồng hồ** trong hệ thống.
- **Bước 2:** Lấy chỉ số của từng đồng hồ trừ đi mức trung bình đó để xem ai đang "vọt" lên quá cao.
- **Kết quả:** Trả về danh sách các đồng hồ có chỉ số Z-Score > 3 (ngghi ngờ quá tải hoặc trộm điện).

3. Công đoạn "Tìm kiếm quanh vị trí sự cố" (Geospatial)

Khi một trạm biến áp nổ, bạn cần biết những ai bị ảnh hưởng:

- **Bước 1 (\$geoNear):** Database sẽ "quét" một vòng tròn trên bản đồ quanh vị trí trạm biến áp.
 - **Bước 2 (\$limit):** Chỉ lấy những hộ dân nằm trong bán kính 2km.
 - **Kết quả:** Danh sách khách hàng cần gửi thông báo mất điện ngay lập tức.
-

Tại sao phải dùng "Dây chuyền" này mà không dùng hàm tìm kiếm thông thường?

- **Tốc độ:** Nếu bạn lấy dữ liệu về máy tính rồi mới tính toán bằng code JavaScript, máy tính sẽ bị treo vì dữ liệu quá lớn.
- **Sức mạnh:** Aggregation Pipeline chạy trực tiếp trên Server của Database (nơi có cấu hình cực mạnh), giúp xử lý hàng triệu bản ghi chỉ trong vài phần nghìn giây.

Tóm lại: Aggregation Pipeline chính là "**bộ não tính toán**" giúp biến những con số vô hồn trong Database thành những thông tin có giá trị trên giao diện Admin của bạn._

2. Geospatial Query (Truy vấn Không gian)

Sử dụng cho tính năng bản đồ và tìm kiếm các sự cố theo vùng địa lý.

- **Cách áp dụng:** Sử dụng Index 2dsphere trên trường location của các đồng hồ. Áp dụng toán tử \$geoNear trong Aggregation Pipeline để tìm các đồng hồ trong bán kính (Radius) nhất định từ một tọa độ.
- **Lợi ích:** Cho phép hệ thống xác định nhanh các hộ dân bị ảnh hưởng khi một trạm biến áp cụ thể gặp sự cố.

4. Materialized Views (Bảng tổng hợp dữ liệu)

Sử dụng cho các báo cáo tháng và thống kê dài hạn.

- **Cách áp dụng:** Schema MonthlyUsageSummary đóng vai trò là một Materialized View. Dữ liệu được tính toán sẵn (Pre-aggregated) từ các bản ghi chi tiết và lưu vào đây.
- **Lợi ích:** Khi Admin xem báo cáo tháng, hệ thống không cần quét hàng triệu bản ghi chi tiết mà chỉ cần đọc vài bản ghi đã tổng hợp, giúp Dashboard tải gần như tức thì.

5. Atomic Updates (Cập nhật nguyên tử)

Sử dụng các toán tử cập nhật mạnh mẽ của MongoDB để tránh xung đột dữ liệu (Race Conditions).

- **Cách áp dụng:** Sử dụng \$push để thêm bản ghi mới, \$inc để cộng dồn tổng tiêu thụ trong ngày, và \$pop để hoàn tác (Undo) dữ liệu.
- **Lợi ích:** Đảm bảo tính nhất quán dữ liệu ngay cả khi có hàng ngàn đồng hồ gửi dữ liệu về cùng một lúc.

6. Indexing Strategy (Chiến lược đánh chỉ mục)

- **Compound Index:** Đánh chỉ mục kết hợp { meter_id: 1, day: -1 } để tối ưu các truy vấn tìm kiếm lịch sử của một khách hàng cụ thể.
- **TTL Index (Dự kiến):** Có thể áp dụng cho các thông báo (Notifications) để tự động xóa các cảnh báo cũ sau 30 ngày, giữ cho hệ thống luôn gọn nhẹ.

Luận điểm B: Tại sao chọn LSM-Tree cho dữ liệu hàng tỷ lượt đọc (Smart Grid)?

Khi đối mặt với câu hỏi: "*Tại sao không dùng SQL Server (B-Tree) để lưu chỉ số điện mà phải dùng MongoDB (LSM-Tree)?*", bạn hãy trả lời như sau:

- **Nhược điểm của B-Tree (SQL truyền thống):** B-Tree duy trì dữ liệu theo thứ tự sắp xếp ngay trên đĩa. Khi có hàng triệu đồng hồ gửi dữ liệu cùng lúc, B-Tree phải thực hiện các thao tác **Ghi ngẫu nhiên (Random I/O)** để chèn dữ liệu vào đúng vị trí của nó trên đĩa. Điều này gây ra hiện tượng phân mảnh và làm đầu đọc ổ cứng phải di chuyển liên tục, dẫn đến tốc độ ghi bị "nghẽn cổ chai" nghiêm trọng khi dữ liệu lớn dần.
 - **Ưu thế của LSM-Tree (MongoDB/NoSQL):** LSM-Tree được thiết kế cho các hệ thống "Write-Heavy" (Ghi nhiều hơn Đọc).
 1. Dữ liệu mới sẽ được ghi vào **MemTable** (Bộ nhớ tạm trên RAM) cực nhanh.
 2. Khi MemTable đầy, nó được ghi xuống đĩa theo dạng **Sequential I/O (Ghi tuần tự)** vào các file gọi là **SSTables**.
 3. Ghi tuần tự luôn nhanh hơn ghi ngẫu nhiên hàng chục lần (giống như việc viết tiếp vào cuối một quyển vở thì nhanh hơn việc tìm đúng trang cũ để chèn thêm một dòng).
 - **Kết luận:** Với hàng tỷ chỉ số điện, LSM-Tree giúp hệ thống của chúng ta duy trì tốc độ ghi ổn định, không bị chậm theo thời gian như các cấu trúc dữ liệu cũ.
-

Luận điểm D: Vai trò của WAL trong việc bảo vệ giao dịch thanh toán

Khi được hỏi: "*Nếu máy chủ thanh toán bị sập nguồn đúng lúc đang trừ tiền khách hàng, làm sao để dữ liệu không bị hỏng?*", bạn hãy trả lời dựa trên cơ chế **WAL (Write-Ahead Logging)**:

- **Nguyên tắc "Ghi nhật ký trước, sửa dữ liệu sau":** Trong các hệ thống tài chính (SQL Server), Database không bao giờ sửa trực tiếp số dư khách hàng trên file dữ liệu chính ngay lập tức (vì ghi file chính rất chậm).
- **Cách WAL hoạt động:**
 1. Mọi thao tác thay đổi (ví dụ: Trừ 100k tiền điện) sẽ được ghi dưới dạng một dòng nhật ký cực ngắn vào file **WAL (Write-Ahead Log)** trước. Ghi vào file này cực nhanh vì nó là ghi nối đuôi (Append-only).

2. Sau khi WAL đã được ghi xuống đĩa cứng an toàn, hệ thống mới bắt đầu cập nhật dữ liệu thật ở file chính.
- **Kịch bản Sự cố:** Nếu máy chủ mất điện đúng lúc đang sửa file chính, khi khởi động lại, Database sẽ thực hiện quy trình **Recovery**:
 - Nó kiểm tra file WAL và thấy có một giao dịch "Trừ 100k" đã được ghi an toàn nhưng chưa kịp cập nhật vào bảng dữ liệu chính.
 - Nó sẽ **Replay** (thực hiện lại) thao tác đó.
 - **Kết luận:** WAL chính là "hộp đen máy bay" của Database. Nó đảm bảo tính **Bền vững (Durability)** của ACID, giúp hóa đơn khách hàng không bao giờ bị mất hoặc sai lệch dù có sự cố vật lý xảy ra.

1. Về CAP Theorem (Chiếm 35% điểm kiến trúc)

Hội đồng sẽ hỏi: "Hệ thống của bạn chọn gì trong 3 chữ C-A-P?". Hãy trả lời như sau:

- **Với SQL Server (Billing/User):** Chúng ta ưu tiên **C (Consistency - Tính nhất quán)**. Khi tính tiền điện hoặc đăng nhập, dữ liệu phải tuyệt đối chính xác và đồng nhất. Chúng ta chấp nhận hệ thống có thể tạm ngưng (giảm A) nếu có sự cố phân mảnh để đảm bảo không bao giờ có 2 kết quả khác nhau cho 1 hóa đơn.
- **Với MongoDB (Usage Data):** Chúng ta ưu tiên **A (Availability) và P (Partition Tolerance)**. Trong hệ thống Smart Grid, việc nhận dữ liệu từ đồng hồ là liên tục. Nếu một node DB bị sập, hệ thống phải vẫn tiếp tục nhận dữ liệu (A) và hoạt động bình thường trên các node khác (P). Chúng ta chấp nhận "Nhất quán sau" (Eventual Consistency) — tức là dữ liệu có thể trễ vài mili giây nhưng không bao giờ được phép từ chối nhận dữ liệu từ đồng hồ.

2. Về Technical Logic (Chiếm 35% điểm kỹ thuật)

Phần này bạn cần chỉ ra được các "vũ khí" hạng nặng:

- **Triggers:** Bạn hãy mở file SQL và chỉ vào **Trigger trg_CriticalPriority**. Hãy giải thích: *"Đây là logic bảo vệ tầng sâu. Ngay cả khi code Node.js bị lỗi, Database vẫn tự động bảo vệ khách hàng bằng cách nâng mức ưu tiên lên CRITICAL khi thấy trạng thái OFFLINE"*.
- **Aggregations:** Hãy giải thích về **Z-Score Pipeline**. Nhấn mạnh rằng bạn dùng \$stdDevPop để tính toán thống kê ngay trong Database thay vì dùng code loop, giúp tốc độ xử lý nhanh gấp hàng trăm lần.
- **Recursive CTE (Nếu hội đồng hỏi):** Bạn có thể đề xuất dùng nó cho việc truy vấn "Cây phả hệ điện" (ví dụ: Từ Trạm tổng -> Trạm nhánh -> Từng hộ dân). *Nếu bạn cần, tôi sẽ viết sẵn cho bạn 1 đoạn code Recursive CTE để demo.*

3. Về AI Audit & Ethics (Chiếm 15% điểm)

Đây là phần "bẫy" để xem bạn có thực sự hiểu code không hay chỉ copy-paste từ AI.

- **Cách trả lời:** *"Tôi dùng AI để gợi ý cấu trúc Pipeline, nhưng tôi đã trực tiếp Audit lỗi **Race Condition** (Xung đột ghi). Ví dụ: Trong lệnh ghi chỉ số, tôi yêu cầu dùng toán tử \$inc của MongoDB để đảm bảo tính **Atomic (Nguyên tử)**. Nếu dùng phép cộng $a = a + b$ ở tầng code, khi có 2 yêu cầu cùng lúc sẽ dẫn đến sai số. Việc dùng \$inc giúp Database tự xếp hàng và cộng dồn chính xác"*.

4. Oral Defense & Failure Paths (Chiếm 15% điểm)

Bạn cần giải thích được: *"Nếu Notification Service bị sập thì sao?"*.

- **Trả lời:** *"Hệ thống sẽ bị mất tính năng thông báo Real-time, nhưng dữ liệu điện vẫn được Usage Service lưu an toàn vào MongoDB. Khi Notification Service sống lại, nó có thể quét lại các bản ghi chưa xử lý để gửi bù"*.

1. Kỹ thuật Gom nhóm sự kiện (Event Aggregation)

Thay vì xử lý mỗi cảnh báo một cách độc lập, hệ thống sẽ nhìn vào **Cửa sổ thời gian (Time Window)** và **Ngữ cảnh (Context)**:

- **Cửa sổ thời gian:** Trong vòng 5 phút gần nhất.
- **Ngữ cảnh:** Cùng một neighborhood_id (Khu vực).
- **Hành động:** Hệ thống đếm xem có bao nhiêu "phiếu bầu" cho sự cố ở khu vực này.

2. Ngưỡng chặn thông minh (Threshold-based Suppression)

Chúng ta thiết lập một con số "Ngưỡng" (Ví dụ: 3 hộ dân).

- **Nếu < Ngưỡng:** Hệ thống coi đây là lỗi cá nhân (Micro Fault). Nó sẽ gửi thông báo riêng lẻ cho từng nhà.
- **Nếu >= Ngưỡng:** Hệ thống ngay lập tức thực hiện lệnh **updateMany** trong MongoDB để chuyển trạng thái tất cả các tin lẻ thành SUPPRESSED (Bị chặn). Sau đó, nó chỉ tạo duy nhất 1 bản ghi MACRO_EVENT (Sự cố diện rộng).

3. Kỹ thuật Xử lý trễ (Debouncing/Delayed Delivery)

Đây là "bí quyết" để hệ thống có thời gian suy nghĩ.

- Mỗi khi có một lỗi lẻ đến, chúng ta không gửi mail ngay mà dùng hàm **setTimeout (chờ 3 giây)**.
- Trong 3 giây đó, nếu có thêm nhiều hộ dân khác cũng báo lỗi, lệnh "Chặn diện rộng" sẽ kịp thực thi.
- Sau 3 giây, thông báo lẻ mới kiểm tra lại: *"Nếu tôi chưa bị ai chặn (vẫn là PENDING), thì tôi mới được phép gửi đi"*.